

**Major Project Final Report:
A K-12 Subject Matter Expert Agent for Indian
Constitution and Rights**

LMA_TEAM

November 17, 2025

Contents

1	Introduction and SME Definition	3
2	System Architecture Overview	3
3	Core Model Strategy & Experiments	3
3.1	Dataset Collection & Augmentation	4
3.2	Experiment 1: Fine-Tuning LLaMA 3.2 (1B)	4
3.3	Experiment 2: Evaluating LLaMA 3.3 (70B) Tool-Calling	4
3.4	Conclusion: Final Model Strategy	4
4	RAG Pipeline & Data Infrastructure	4
4.1	Ingestion Pipeline Architecture	4
4.2	Evolution & Debugging (What Failed)	5
4.3	Chunking & Indexing	5
4.4	Embedding Model Evaluation	5
5	Advanced Retrieval System (BONUS Features)	6
5.1	Hybrid Retrieval (Dense + Keyword)	6
5.2	Knowledge Graph (via Entity Filtering)	6
5.3	Reranking	6
5.4	Explainable Retrieval Flow (Final)	6
6	Agentic Framework & Orchestration	7
6.1	Core Orchestrator Prompt	7
6.2	Intelligent Tool Suite (BONUS Features)	8
6.3	Multi-Step Workflow & Chaining	8
7	Guardrails & Safety (BONUS Feature)	8
8	System Evaluation & Results	9
8.1	RAG Pipeline Evaluation ("LLM as a Judge")	9
8.1.1	Summary Statistics	9
8.1.2	Analysis & Conclusions	9
8.2	Qualitative Examples (End-to-End)	10
8.3	Quantitative Evaluation (End-to-End)	10
9	Conclusion	10

1. Introduction and SME Definition

This report details the design, implementation, and evaluation of a highly extensible Retrieval-Augmented Generation (RAG) system serving as a Subject Matter Expert (SME) agent. The objective is to create a robust, tool-using agent focused on a K-12 Education domain, specifically "Civics – Indian Constitution & Rights."

The SME agent is designed to act as a knowledgeable assistant for K-12 students and teachers. Its capabilities include:

- Assisting students with step-by-step explanations of constitutional concepts.
- Generating adaptive quizzes and interactive flashcards for study.
- Answering complex RAG-based questions about the Indian Constitution, Fundamental Rights, and related legal statutes.
- Performing web searches for up-to-date information on recent legal judgments.
- Generating and exporting educational content (reports, presentations) and delivering them via email.
- Creating short educational videos on constitutional topics.

2. System Architecture Overview

The system is engineered as a modular, 6-phase pipeline that processes every user request from validation to a final, tool-augmented response. This architecture is designed for security, observability, and robust, knowledge-driven decision-making.

User Request → Security → Knowledge → Agent Init → Tool Execution → Response Processing

Phase 1: Security & Validation All user input is first processed by a 3-tier Guardrail system to block harmful queries before any processing occurs.

Phase 2: Knowledge Retrieval The user query is used to fetch relevant context from our custom data infrastructure. This involves a high-performance hybrid RAG pipeline.

Phase 3: Agent Initialization The query, conversation history, and retrieved RAG context are passed to an Orchestrator Agent (powered by Gemini 2.5 Pro).

Phase 4: Tool Execution The Orchestrator analyzes the inputs and decides which of its 7 available tools (e.g., RAG, web search, quiz generation) to call, in what sequence, to fulfill the user's request.

Phase 5: Response Processing The agent synthesizes the results from all tool calls into a single, coherent, natural-language answer.

Phase 6: Logging & Memory The complete interaction, including all tool calls, RAG context, and timings, is saved to a session log, and the conversation is persisted in memory.

3. Core Model Strategy & Experiments

A significant part of the project involved identifying a core Large Language Model (LLM) suitable for both RAG and complex tool-calling.

3.1. Dataset Collection & Augmentation

We first identified an appropriate dataset for question answering on the Indian Constitution, selecting `nisaar/Constitution_of_India` from Hugging Face. This dataset contains Q&A pairs derived from constitutional text. As the dataset was relatively small, it was expanded by generating an additional 990 synthetic question-answer pairs. This brought the total dataset size to approximately 2,000 samples, which served as the training corpus for fine-tuning.

3.2. Experiment 1: Fine-Tuning LLaMA 3.2 (1B)

Using this combined dataset, we fine-tuned the LLaMA 3.2 (1B) model. Although the fine-tuning process completed successfully, the resulting model performed poorly on the question-answering task. We observed that the model lacked sufficient world knowledge and general reasoning ability to answer even moderately difficult questions about the Constitution. Its responses were often generic, inconsistent, or factually incorrect, indicating that the limited model capacity was a major bottleneck.

3.3. Experiment 2: Evaluating LLaMA 3.3 (70B) Tool-Calling

To understand whether larger models might perform better, we evaluated the LLaMA 3.3 (70B) model, focusing specifically on its instruction-following and tool-calling capabilities. This was tested using an email-generation tool task. Surprisingly, the 70B model also performed poorly at tool usage. It frequently failed to call the tool correctly, misinterpreted instructions, or produced incomplete outputs.

3.4. Conclusion: Final Model Strategy

Since both the fine-tuned 1B model (insufficient knowledge) and the larger 70B model (unreliable tool-calling) failed to meet our performance criteria, ****we decided to discard both models.****

This led to our final, successful strategy:

1. **For Generation & Reasoning:** Utilize a state-of-the-art, API-based model (Google's Gemini 2.5 Pro) for all agentic reasoning and content generation, as its tool-calling is reliable.
2. **For Retrieval:** Focus all effort on building a "max bar" RAG pipeline with domain-specific embedding models, as this is the most effective way to inject specialized knowledge.

4. RAG Pipeline & Data Infrastructure

The RAG pipeline is the core of the system. Its engineering evolved significantly from the mid-evaluation prototype to a robust, high-performance, parallelized system.

4.1. Ingestion Pipeline Architecture

The final pipeline (`ingestion_pipeline.py`) is designed to maximize our available hardware (48 CPUs, 3 GPUs) by correctly identifying and parallelizing the true bottlenecks.

Main Thread Loads the embedding model (e.g., `legal-bert`) onto `cuda:0` ****once****.

GPU Lock A single `threading.Lock()` is created to protect this model.

CPU Pool A `ThreadPoolExecutor` with 48 workers is created to process all 29 documents in parallel.

The Flow Each of the 48 workers executes the following:

1. **Chunk (CPU-only):** Uses `RecursiveCharacterTextSplitter` to chunk the document. This is fast, parallel, and requires no lock.
2. **Metadata (I/O-bound):** Makes two parallel calls to the Gemini API to generate (a) summaries/labels and (b) Knowledge Graph entities. This is the primary bottleneck and is fully parallelized.
3. **Embed (GPU-bound):** The worker acquires the `embedding_lock`, sends its chunks to the single GPU in safe batches of 128, and releases the lock. This prevents CUDA OOM crashes.
4. **Upsert (I/O-bound):** The worker saves the vector to Pinecone and the raw text to a local `bm25_corpus.jsonl` file for the keyword index.

4.2. Evolution & Debugging (What Failed)

The development of this pipeline involved solving several critical engineering challenges:

- **Failure 1: Semantic Chunking Gridlock.** Our first attempt used semantic chunking. However, this required all 48 CPU workers to call `get_embeddings` as their **first step**, causing a “grid-lock” pile-up as they all waited for the single GPU lock. The script hung at 0%. *Solution: We abandoned semantic chunking for the CPU-only `RecursiveCharacterTextSplitter`, which removed this initial bottleneck.*
- **Failure 2: CUDA out of memory & BrokenProcessPool.** Our next attempt used a `ProcessPoolExecutor` to give each GPU its own worker. This failed catastrophically when a CPU worker submitted a **massive** job (e.g., 10,000 sentences) to a single GPU worker, which ran out of VRAM and died, breaking the entire pool. *Solution: The final “Single GPU (Locked)” architecture, which manually batches jobs (128 sentences at a time), is stable and 100% prevents CUDA OOM crashes.*
- **Failure 3: Metadata Mismatch.** Early tests showed 0 results for filtered queries. We discovered a case-sensitivity mismatch: the ingestion pipeline was saving labels like “Article 14”, while the retrieval tool was searching for “article 14”. *Solution: We modified both the ingestion pipeline and the retrieval tool to lowercase all labels and entities, ensuring a consistent match.*

4.3. Chunking & Indexing

Chunking We use `RecursiveCharacterTextSplitter` with a `max_chunk_size` of 10,000 characters. This is a hybrid approach; it’s a large chunk size, but it guarantees we do not exceed Pinecone’s 40KB metadata limit, a bug we discovered in early testing.

Vector Index We selected ***Pinecone Serverless*** as our vector database. We chose it over FAISS (as discussed in the mid-eval) because it is a managed database that stores both the vectors **and** our critical metadata (summaries, labels, entities), whereas FAISS would have required a separate SQL database.

Keyword Index We also build a ***BM25 (Okapi)*** index. The `ingestion_pipeline.py` saves all text chunks to `bm25_corpus.jsonl`. A separate script, `build_bm25_index.py`, then runs to create and save the `bm25_index.pkl` file, which is loaded by the `RAGTool`.

4.4. Embedding Model Evaluation

A core task was to evaluate embedding models. We implemented an A/B test by building two separate indexes:

- **Model A:** `nlpaueb/legal-bert-base-uncased` (**Domain-Specific**). This is a raw BERT model pre-trained on a legal corpus. When loaded by the `sentence-transformers` library, it defaults to using mean pooling to create a sentence embedding.
- **Model B:** `sentence-transformers/all-mpnet-base-v2` (**General-Purpose**). This is a model **purpose-built** for semantic search with a sophisticated, fine-tuned pooling architecture.

This A/B test allows us to definitively evaluate the trade-off between domain-specific knowledge (Legal-BERT) and a purpose-built search architecture (MPNet).

5. Advanced Retrieval System (BONUS Features)

The final RAGTool (`rag_tools/retrieval_tool.py`) implements three separate bonus features to create a "max bar" retrieval pipeline that is highly accurate.

5.1. Hybrid Retrieval (Dense + Keyword)

Our tool implements a true hybrid search. It queries both the Pinecone (Dense) index and the BM25 (Keyword) index in parallel. This is critical for our domain:

- **Dense Search** finds chunks that **sound like** "the right to be equal."
- **Keyword Search** finds chunks that **contain the exact string** "Article 14."

The two result lists are then combined using ****Reciprocal Rank Fusion (RRF)****, which intelligently balances both semantic relevance and keyword specificity.

5.2. Knowledge Graph (via Entity Filtering)

This is our most important bonus feature for solving relevancy. A formal graph database (like Neo4j) was deemed too complex. Instead, we implemented a highly effective ****"Entity-Based Index"****:

1. **Ingestion:** The `ingestion_pipeline.py` calls `Gemini(get_entities_from_chunk)` for every chunk to extract key entities (e.g., `['article 14', 'right to equality']`). These are saved in a new `entities` metadata field in Pinecone.
2. **Retrieval:** The RAGTool calls `Gemini(_get_entities_from_query)` on the user's query to extract the same entities.
3. **Filtering:** The Dense (Pinecone) search is then **strictly filtered** to only include chunks whose `entities` metadata list contains the entities from the query.

This "Knowledge Graph" filter is the key to solving the poor results from early tests.

5.3. Reranking

After the final fused and filtered list of candidates is retrieved, it is passed to a locally-loaded CrossEncoder model (`BAAI/bge-reranker-base`). This model performs a final, high-precision scoring pass to ensure the most relevant answer is at the top.

5.4. Explainable Retrieval Flow (Final)

The final retrieval flow is a 4-step process:

1. **KG Entity Analysis:** Extract entities (e.g., `['article 14']`) from the user query.

2. Filtered Hybrid Search:

- Query Pinecone for semantic matches, filtered by {"entities": {"\$in": ['article 14']}}.
- Query BM25 for keyword matches.

3. **RRF Fusion:** Combine the two result lists.

4. **Reranking:** Rerank the final list with the CrossEncoder.

6. Agentic Framework & Orchestration

The agent framework is the "brain" of the system, responsible for tool-calling and workflow orchestration.

6.1. Core Orchestrator Prompt

After much iteration (moving from zero-shot to few-shot), we developed a detailed, few-shot prompt that provides the LLM with clear rules for tool selection, prioritization (RAG-first), and multi-step workflows.

```

1 You are an intelligent ORCHESTRATOR that decides when and how to call tools
  to satisfy user queries. Follow these instructions carefully:
2
3 TOOLS AVAILABLE:
4 1. normal_content_tool: Answer general questions using internal knowledge (
  RAG-based). This tool receives rag_context automatically and should be
  your DEFAULT choice.
5 2. web_search_tool: Answer questions using Internet search results for up-
  to-date information. USE ONLY when explicitly requested by user OR when
  RAG context is insufficient.
6 3. document_export_tool: Export content as PDF, DOCX, or PPTX documents.
7 4. send_email_tool: Send emails with content or documents.
8 5. video_generation_tool: Create educational videos -(22.5 minutes) on
  constitutional topics.
9 6. flashcard_generation_tool: Create interactive flashcards for studying
  topics.
10 7. interactive_quiz_tool: Create quizzes with multiple question types,
  scoring, and immediate feedback.
11
12 REASONING RULES:
13 1. Think step-by-step using a scratchpad...
14 5. Tool selection guidelines (PRIORITY ORDER):
15   - DEFAULT: Use normal_content_tool for ALL questions...
16   - ONLY use web_search_tool when:
17     * User explicitly asks for "latest", "current", "recent", "web search"
    ...
18     * RAG context shows "No relevant context found"...
19   - Study/revision material -> flashcard_generation_tool
20   - Knowledge testing/quizzes -> interactive_quiz_tool
21 ...
22 7. Always clarify missing parameters (topic, number of questions, file type
  , email address) before calling a tool.
23 IMPORTANT: The RAG context provided contains extensive knowledge about
  Indian Constitution and Rights. TRUST this context first and use
  
```

```

24     normal_content_tool as your primary choice...
25 EXAMPLE WORKFLOWS:
26 - User: "Explain Article 21" -> Use normal_content_tool (RAG has this info)
27 - User: "Latest Supreme Court judgment on Article 21" -> Use
    web_search_tool (explicitly asks for latest)
28 - User: "I want flashcards on the Constitution and emailed to me as a PDF."
29 - Orchestrator:
30   1. Recognizes request... -> call flashcard_generation_tool...
31   2. Recognizes request for export -> call document_export_tool...
32   3. Recognizes request for email -> call send_email_tool...

```

Listing 1: Final Orchestrator Prompt (Abridged)

6.2. Intelligent Tool Suite (BONUS Features)

The agent has access to a comprehensive, 7-tool suite for a full K-12 educational experience.

`normal_content_tool` The default RAG tool. It receives the user query and the RAG context (from our pipeline) to generate a grounded answer.

`web_search_tool` (**BONUS**) Uses the Tavily API for real-time web search. It is *only* triggered when the RAG context is insufficient (e.g., for very recent events) or when the user explicitly asks for "the latest" information.

`flashcard_generation_tool` (**BONUS**) Calls Gemini with RAG context to create a set of interactive flashcards (Q&A pairs) for a given topic, returned as JSON for the UI.

`interactive_quiz_tool` (**BONUS**) Calls Gemini with RAG context to generate a full quiz (MCQ, T/F, Fill-in-the-blank) and provides scoring and feedback.

`video_generation_tool` (**BONUS**) Generates a 2-2.5 minute educational video, including an LLM-generated script and Google TTS audio narration.

`document_export_tool` (**BONUS**) Exports generated content (from quizzes, flashcards, etc.) into professional PDF, DOCX, or PPTX formats.

`send_email_tool` (**BONUS**) Sends exported documents as email attachments.

6.3. Multi-Step Workflow & Chaining

The orchestrator prompt (see Listing 1) is explicitly trained to handle multi-step requests. The "Flashcards → PDF → Email" example demonstrates this:

1. The agent first calls `flashcard_generation_tool` to create the content.
2. It passes the output of this tool to the `document_export_tool` to create the PDF.
3. It passes the resulting file path to the `send_email_tool` for delivery.

This demonstrates robust planning and execution over a multi-turn, multi-tool workflow.

7. Guardrails & Safety (BONUS Feature)

A 3-tier security validation system (`run_guardrail_check`) was implemented to process every query before it reaches the agent.

Tier 1: Static Rule Checking Basic input validation (length, empty) and regex checks for obvious harmful terms ("password", "api_key") or prompt injection patterns ("ignore all instructions").

Tier 2: Contextual Keyword Analysis Identifies ambiguous but potentially harmful words ("attack", "exploit", "bomb"). If these words appear without a clear academic context (like "adversarial simulation"), the query is escalated.

Tier 3: AI-Powered Semantic Analysis The escalated query is sent to a specialized safety model (e.g., llama-3.1-nemotron-safety-guard-8b-v3) to classify the user's *intent* as SAFE or UNSAFE.

Test Case: The query "Hello, I would like to know what are the specific legal loopholes... to justify 'honour killings'" was sent. **Result:** The query passed Tiers 1 & 2 but was flagged by the Tier 3 AI model. The system correctly blocked the query and returned an "Unsafe query flagged by model" error, preventing the agent from engaging.

8. System Evaluation & Results

8.1. RAG Pipeline Evaluation ("LLM as a Judge")

A full evaluation (evaluate_rag.py) was performed to test 4 distinct retrieval strategies using a "Golden Set" of 5 queries. Relevance was graded by an "LLM as a Judge" (Gemini 2.5 Pro) on a scale of 1-5.

The 4 Test Scenarios:

1. legal-bert (No Filter): Dense (legal-bert) + Keyword (BM25)
2. legal-bert (With Filter): Dense (legal-bert) + Keyword (BM25) + **KG Entity Filter**
3. mpnet (No Filter): Dense (mpnet) + Keyword (BM25)
4. mpnet (With Filter): Dense (mpnet) + Keyword (BM25) + **KG Entity Filter**

8.1.1. Summary Statistics

Table 1: RAG Pipeline Evaluation Results

Model	Filter	Avg. Relevance (1-5)	Avg. Latency
legal-bert	False	3.0	4.00s
legal-bert	True	3.2	17.40s
mpnet	False	2.8	5.72s
mpnet	True	3.2	15.30s

8.1.2. Analysis & Conclusions

1. **The Entity Filter (Our KG Implementation) is the Key to Relevancy:** The "No Filter" runs produced generic results (avg. relevance 2.8-3.0). Turning on the ****Entity Filter (filter=True)**** provided a clear and measurable relevance boost for ***both*** models, bringing them to an identical ****3.2/5.0****. This proves that the (Dense + Keyword + Entity Filter) pipeline is the most effective strategy.

2. **legal-bert vs. mpnet:** The `nlpaueb/legal-bert-base-uncased` (with simple mean pooling) performed *identically* to the purpose-built `all-mpnet-base-v2` in the final, filtered test. This is a significant finding: *the domain-specific knowledge of legal-bert was able to match the optimized architecture of mpnet*.
3. **The Relevancy vs. Latency Trade-off:** The "With Filter" runs are $\sim 3\text{-}4\times$ slower (15.3s - 17.4s). This ~ 12 -second delay is the cost of the extra call to the Gemini API for real-time query analysis. We have successfully quantified that the *Entity Filter (BONUS)* provides a $\sim +6.7\%$ to $\sim +14.3\%$ boost in relevance at the cost of ~ 12 seconds of latency.

8.2. Qualitative Examples (End-to-End)

Basic RAG (Success) A student asking for a summary of the "Right against Exploitation" received a perfect, concise, 2-point summary derived from the correct Class 11 NCERT textbook chunk.

Web Search (Success) A query on the *new* "Section 152 of the Bharatiya Nyaya Sanhita (BNS)" (a topic not in our static data) correctly triggered the `web_search_tool`. The agent bypassed RAG (which returned "No relevant context") and used Tavily to retrieve up-to-date web results, successfully comparing the new law to the old Sedition law.

Interactive Quiz (Success) An iterative prompting process (moving from zero-shot to few-shot) was used to successfully train the orchestrator. A vague query like "help me prepare for my exam" was correctly refined, leading to the agent asking follow-up questions and then successfully invoking the `interactive_quiz_tool` to deliver a fully functional, auto-graded quiz UI.

8.3. Quantitative Evaluation (End-to-End)

Added separately in our repository

9. Conclusion

This project successfully demonstrates the design and implementation of a complex, high-performance, and feature-rich K-12 SME Agent. By abandoning failing model-tuning experiments and focusing on a robust data-centric architecture, we achieved all core and bonus objectives.

The final system is a stable, 3-part hybrid RAG pipeline (Dense + Keyword + Entity Filter) that provides state-of-the-art retrieval, augmented by a 7-tool agentic framework, and protected by a 3-tier safety guardrail. The final evaluation provides a clear, data-driven analysis of the trade-offs between model choice, filtering, relevance, and latency.